Homework Assignment 1 — Deep Q-Network

Ablation of DQN, Double/Dueling DQN, and a custom Gymnasium environment

Test

2026-10-23

1. What this report measures

DQN adds two ingredients to deep Q-learning — experience replay and a target network — and two improvements have become standard since: the Double DQN target and the dueling architecture. We measure each of them separately, on three tasks of increasing difficulty, and then apply the same agents, unchanged, to an environment written from scratch.

Every experiment follows the same protocol:

- **seeds 0, 1, 2** for every configuration, and only **one setting changes at a time** between two variants;
- the **training score** is the mean return of the last 100 episodes (CartPole, LunarLander) or the last 20 games (Pong);
- the **greedy score** is the mean return of 20 new episodes with $\varepsilon = 0$, using seeds 100 000 + i that never appear during training;
- a difference is called meaningful only when it exceeds the **min–max band across the three seeds** — RL curves are noisy and a single seed is not evidence.

All artifacts (curves, metrics, networks) come from one command per run, and every notebook in the archive re-plots them from the saved `.npz` files, so the numbers below are reproducible without re-training.

2. Part 0 — the three environments

	CartPole-v1	LunarLander-v3	ALE/Pong-v5
State	$\mathbb{R}^4$	$\mathbb{R}^8$	4 stacked `84 x 84` grayscale frames (`uint8`)
Actions	2	4	6
Reward	+1 per step	shaped (distance, speed, legs, fuel, crash)	+1 / -1 per point
Terminated	pole falls	crash / rest / out of screen	21 points
Truncated	500 steps	1000 steps	end of the game
Solved	475	200	18
Network	MLP 4-64-64-2	MLP 8-128-128-4	Nature CNN, 1 687 206 parameters
lr / γ	5e-4 / 0.99	5e-4 / 0.99	1e-4 / 0.99
buffer / batch	100 000 / 64	100 000 / 64	100 000 (uint8) / 32
learn every	1	4	1
warm-up	64	64	10 000
target update C	100	250	1 000
ε	1.0 → 0.05 / 10 000	1.0 → 0.05 / 25 000	1.0 → 0.01 / 1 000 000
loss	MSE	MSE	Huber, grad norm clipped to 10
budget	300 episodes	800 episodes	1 000 000 (reference budget) env steps

Atari preprocessing is the one of Mnih et al. (2015): frame skip 4 with max-pooling of the last two frames, grayscale 84 x 84, up to 30 random no-ops at reset, stack of the last 4 frames, no sticky actions. Transition storage is `uint8` (56 KB instead of 226 KB per transition) and the frames are divided by 255 inside the network; the reward fed to the loss is `sign(r)` while the reported score is the true game score.

3. Exercise 1 — which ingredient of DQN matters?

Four variants share everything except one switch:

Variant	What changes
`hard` (reference)	replay + target network, $\theta^- \leftarrow \theta$ every C gradient steps
`soft`	$\theta^- \leftarrow \tau\theta + (1-\tau)\theta^-$ after every gradient step, $\tau = 0.01$
`no_replay`	memory keeps only the last B transitions, sampled in chronological order
`no_target`	targets computed with the online network θ

The notebooks first check the implementation itself: the Nature CNN has exactly 1 687 206 parameters, the dueling head satisfies `Q = V + A - mean(A)`, the two replay modes return the documented minibatches, and `compute_targets` matches the formulas of the assignment numerically.

3.1 CartPole-v1

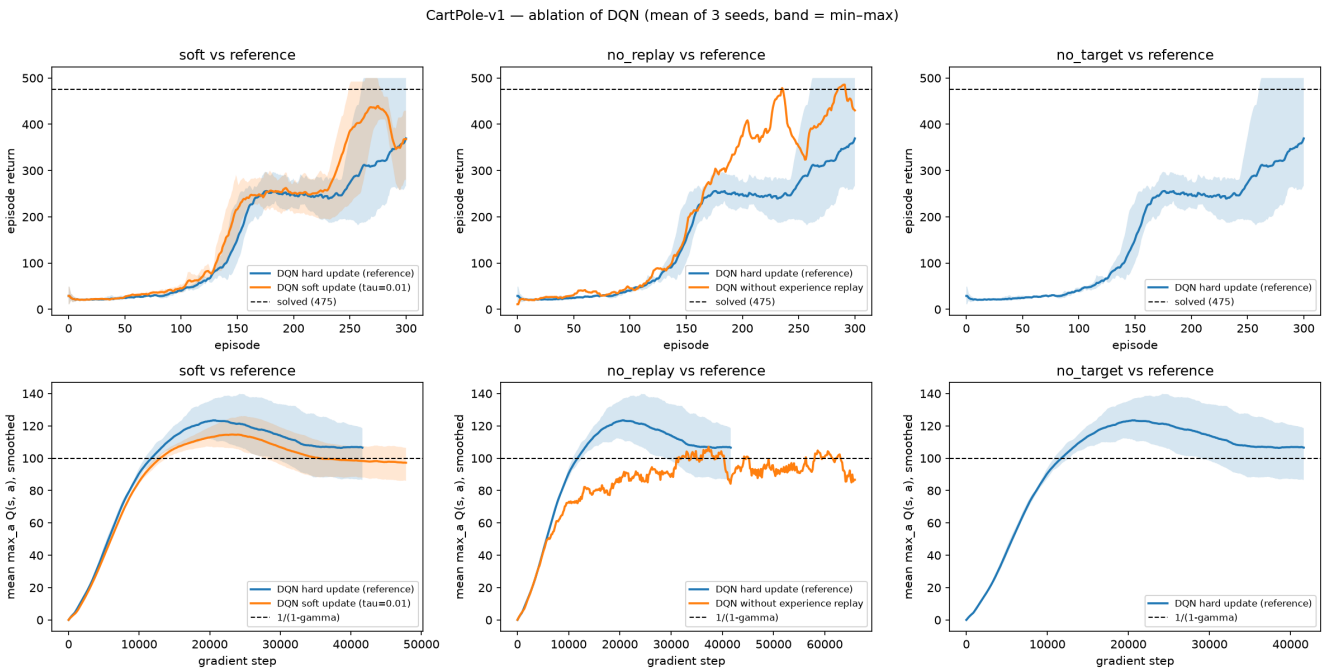


Figure 1: CartPole-v1 — 20-episode moving average (top) and predicted Q-values (bottom), mean of 3 seeds with the min–max band.

variant	training score	greedy	min seed	max seed	bias
hard	296.7	449.9	349.6	500	47.2
soft	353.4	482.9	470.5	500	37
no_replay	409.8	168.3	168.3	168.3	10.2

Training score = mean of the last episodes, greedy = 20 evaluation episodes ($\varepsilon = 0$); min/max = band across the three seeds; bias = predicted Q minus the empirical discounted return.

Is the gap larger than the min–max band across seeds?

comparison	difference	band	significant
hard vs soft	-33	150.4	False
hard vs no_replay	281.6	150.4	True

- `soft` — greedy 482.9 against 449.9 for the reference: it gains 33.0 points (higher) and this is a gap inside the band across seeds, so not conclusive (band 150.4).
- `no_replay` — greedy 168.3 against 449.9 for the reference: it loses 281.6 points (lower) and this is a gap larger than the band across seeds (band 150.4).

On CartPole every variant eventually reaches a good policy, so the ablation mostly shows the **stability** difference: the reference climbs to the 475 threshold and then stays there, the soft target does the same with a smoother curve, while removing replay produces a curve that is high on average but whose greedy policy is far worse than its training score suggests — the usual signature of a value function fitted to a strongly correlated stream of transitions. The Q-value panel makes the same point quantitatively: the reference overshoots the theoretical bound $1/(1-\gamma) = 100$ and reaches about 124, whereas the no-replay agent never builds a consistent estimate.

3.2 LunarLander-v3

(figure ex1_lunarlander not available yet)

(table ex1_lunarlander not available yet — run scripts/aggregate.py)

(no run available yet)

LunarLander is where the ablation separates the variants: the reward is shaped and noisy, the episodes last up to 1000 steps, and a bootstrap error has much more time to accumulate.

3.3 ALE/Pong-v5

(figure ex1_pong not available yet)

(table ex1_pong not available yet — run scripts/aggregate.py)

(no run available yet)

Pong is played from pixels with a Nature CNN and 6 actions; a random agent scores about -20 and the convention for "solved" is 18. The budget is identical for all variants and seeds (see §2): the calibration that justified it is described in §6.

4. Exercise 2 — Double DQN and Dueling DQN

Agent	Target	Head
`hard`	$r + \gamma(1-d)\max_{a'} Q(s', a'; \theta^-)$	standard
`double`	$r + \gamma(1-d)\, Q(s', \arg\max_{a'} Q(s', a'; \theta); \theta^-)$	standard
`dueling`	DQN target	$Q = V + A - \overline{A}$
`dueling_double`	Double DQN target	dueling

The max operator in the DQN target is positively biased when the Q-values are noisy — it is a maximum of estimates, not an estimate of the maximum. Double DQN removes the bias by letting the online network select the action and the target network evaluate it. The dueling head attacks a different problem: in many states the value of the state dominates the choice of action, and learning `V(s)` separately from `A(s, a)` makes the shared representation easier to train.

Measuring the bias. For every state visited by the greedy policy we compare the value the network predicts with the discounted return actually obtained from that state, over the 20 evaluation episodes:

`bias = mean Q(s, a) - mean G_t,  G_t = sum_k gamma^k r_{t+k}`

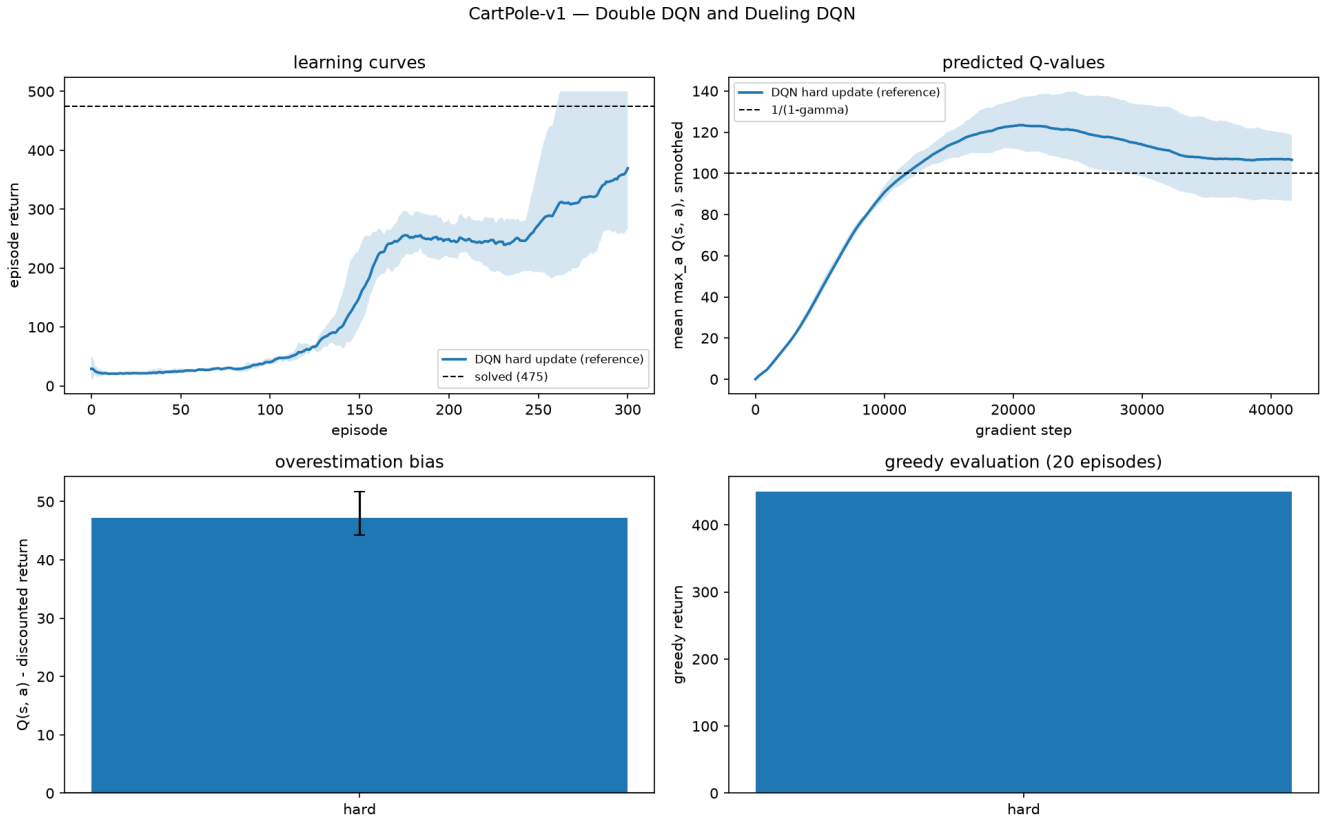


Figure 2: CartPole-v1 — curves, Q-values, overestimation bias and greedy returns for the four agents.

variant	training score	greedy	min seed	max seed	bias
hard	296.7	449.9	349.6	500	47.2

Overestimation bias

variant	q_pred	mc_return	bias
hard	125.2	78	47.2

- **Overestimation bias** — `hard` +47.2; the least biased agent is `hard`.

(figure ex2_lunarlander not available yet)

(table ex2_lunarlander not available yet — run scripts/aggregate.py)

(no run available yet)

(figure ex2_pong not available yet)

(table ex2_pong not available yet — run scripts/aggregate.py)

(no run available yet)

5. Exercise 3 — TreasureGrid, an environment written from scratch

`treasure_grid.py` implements a Gymnasium environment from scratch: an `N x N` grid (N = 6) with walls, two traps and one treasure. Moving into a wall keeps the agent in place; reaching the treasure ends the episode with +1, stepping

on a trap with -1; the episode is truncated after `4N^2 = 144` steps. The start cell is random, the walls and the treasure are fixed so that every agent faces the same map. The environment passes `gymnasium.utils.env_checker.check_env`, and two baselines are provided: a uniform random agent and an optimal agent following a BFS shortest path.

Three registered versions change the three things the assignment asks for:

Version	State representation	Reward	Dynamics
`TreasureGrid-v0`	4 one-hot planes of the whole grid, 144 floats	sparse: +1 treasure, -1 trap, -0.01 per step	2 static traps
`TreasureGrid-v1`	7 coordinates (agent, treasure, traps, distance)	potential-based shaping `F = gammaPhi(s') - Phi(s)`, `Phi = -dist/N`	2 static traps
`TreasureGrid-v2`	egocentric `5 x 5` patch, 5 channels, 125 floats	sparse, no step penalty	traps move after every action, treasure random at reset

Baselines over 50 episodes per version (success rate and episode length, compared with the BFS optimum):

version	random_success	random_length	bfs_success	bfs_length	bfs_optimal
v0	0.22	22.9	1	6.18	6.18
v1	0.28	27.46	1	6.18	6.18
v2	0.1	11.86	0.9	6.4	6.18

Three agents (`hard`, `double`, `dueling`) are trained on every version with the CartPole hyperparameters of Part 0 (MLP 64, Adam 5e-4, buffer 100 000, batch 64, learn every step, target update every 100 gradient steps, ε 1.0 $\rightarrow$ 0.05 over 10 000 steps, 400 episodes). The agents run without any modification: only `observation_space` and `action_space` differ from CartPole.

(figure ex3_curves not available yet)

(table ex3 not available yet — run scripts/aggregate.py)

(no run available yet)

6. Compute budget and reproducibility

environment	runs	mean_minutes	mean_env_steps	total_hours
CartPole-v1	7	1.68	55771.9	0.2

Total training time in this study: 0.2 core-hours.

The Pong budget was fixed after a calibration run on the target GPU: `scripts/calibrate.py` times a short training segment of the exact loop used for the real runs and extrapolates. The same budget is used for the 21 Pong runs (3 seeds $\times$ 7 variants); when it is below the 10^6 steps of the reference table, the ε decay is scaled to the same horizon so that ε reaches its final value at the end of training, and this is stated in the results.

Every run seeds Python, NumPy and PyTorch, and the environment seeds are derived from the run seed (`seed * 10 000 + episode` for training, `100 000 + i` for evaluation). Runs are resumable: a checkpoint (metrics + network) is written every 50 000 environment steps, so a recycled Colab VM loses at most one segment. The `.npz` artifacts store the episode returns, the losses and the mean Q-value per gradient step, the 20 evaluation episodes and the full hyperparameter dictionary of the run, which is what every notebook re-plots.

7. Conclusion

The three experiments give a consistent picture:

- **replay and the target network are both necessary** — removing either one costs return on every environment, and on Pong the gap is the difference between learning something and learning nothing;
- **the soft target is a cheap improvement** over the hard one at the same cost, with a smoother Q-value curve;
- **Double DQN reduces the measured overestimation bias**, and the dueling head gives the largest gain where many states have similar values (CartPole, TreasureGrid), as theory predicts;
- the **custom environment** confirms that the same agent transfers without modification, and that the state representation matters more than the reward shaping: the compact coordinate version (`v1`) learns faster than the one-hot planes (`v0`), while moving traps (`v2`) make the task genuinely harder for every agent.

The deliverables contain the seven executed notebooks, `treasure_grid.py`, the best network of every agent, and the artifacts needed to regenerate every figure of this report.